

1. The Big Picture

The Smart Network Anomaly Analyzer watches network traffic and flags packets that look suspicious, without ever being told in advance what an "attack" looks like. It records network packets (either live from a network card, or replayed from a saved capture file), groups them into short time-windows per conversation ("flow"), turns each window into a set of numbers (a feature vector), and feeds those numbers into a machine-learning model called an Isolation Forest. The model first watches a chunk of ordinary traffic to learn what "normal" looks like, and afterwards scores every new window on how unusual it is compared to that baseline. When a window scores high enough, the system raises an alert, saves the exact packets that caused it as evidence, and shows everything on a web dashboard where you can review, mark as observed, or delete alerts.

The project has two halves that talk to each other over HTTP:

- **Backend (Python)** — a pipeline that reads packets, turns them into features, scores them with a model, and stores the results in a small SQLite database. It also exposes this all as a REST API built with FastAPI.
- **Frontend (React + TypeScript)** — a single-page web app (the dashboard) that calls that API to show live status, let you start/stop a capture session, and browse/manage alerts.

Think of the backend as a factory line and the frontend as the factory's control-room window:

1. **Capture** — packets come in, either from a live network interface or a .pcap file.
2. **Demultiplex** — packets are sorted into "flows" (conversations between two endpoints), regardless of which direction a packet travelled.
3. **Window** — each flow's packets are grouped into batches (either 50 packets or 30 seconds, whichever comes first).
4. **Extract features** — each finished window is boiled down to 14 numbers describing its traffic pattern (packet count, average size, ratio of TCP/UDP/ICMP, SYN/ACK/FIN/RST flag ratios, timing statistics, etc.).
5. **Bootstrap / train** — the first 100 windows across all flows are used to teach the Isolation Forest what ordinary traffic's numbers look like. No scoring or alerting happens yet.
6. **Score** — every window after that is compared against the trained model and gets an anomaly score from 0 (perfectly normal) to 1 (extremely unusual).
7. **Alert** — if a score crosses a threshold (0.80 by default), an alert is created, its severity is graded (low/medium/high/critical), the exact packets are saved as an evidence .pcap, and the alert is written to the database.
8. **Serve** — the FastAPI backend exposes status, capture control, and the alert history as JSON endpoints.
9. **Display** — the React dashboard polls and calls those endpoints so a human can watch it happen and manage the alert history.

2. Backend Walkthrough (Python)

2.1 Ingestion — where packets come from

Everything starts with a `PacketSource`: an abstraction with exactly two implementations.

- **LivePacketCapture** — sniffs packets straight off a real network interface (uses Scapy/Npcap under the hood), optionally filtered by a BPF filter string (like `tcp or port 443`) and optionally in promiscuous mode.
- **PcapFileSource** — replays packets from a saved `.pcap/.pcapng` file on disk, in the order they were recorded, as if they were arriving live. This is what the test files (like `mixed_traffic_test.pcap`) are used for — it lets you replay a controlled, repeatable scenario instead of needing real live traffic.

Both wrap each raw packet in a `PacketEnvelope`: the packet itself, a timezone-aware capture timestamp, where it came from, and a sequence number unique to that capture session. This envelope is the one shape every later pipeline stage understands, so the rest of the code never needs to know whether a packet came from a live card or a file.

2.2 Stream demultiplexing — grouping packets into conversations

A single "conversation" (called a **flow**) between two computers produces packets travelling in both directions — e.g. your laptop sending a request, the server sending a reply. The `StreamDemultiplexer` gives every packet a **StreamKey**: (`protocol`, `endpoint_a`, `endpoint_b`), where the two endpoints are always sorted into the same order regardless of which one sent a given packet. That means a request from `10.0.0.5:51000 → 93.184.216.34:443` and the server's reply `93.184.216.34:443 → 10.0.0.5:51000` land in the exact same flow bucket. This is what lets the pipeline treat a whole back-and-forth conversation as one thing to analyze, instead of two unrelated halves.

It also tracks activity per flow (first seen / last seen / packet count) so idle flows can be expired, and can cap the number of simultaneous flows it tracks (evicting the least-recently-active one) so a burst of traffic — like a port scan, which creates thousands of tiny one-off flows — can't exhaust memory.

2.3 Windowing and feature extraction — turning packets into numbers

A model can't look at raw packets; it needs numbers. So packets belonging to one flow are accumulated into a **window**, which closes as soon as either is true: it has collected **50 packets**, or **30 seconds** have passed since the window started. (Both numbers are configurable.) A window that never fills up — like a single lonely SYN packet from a port scan probe that gets no reply — is still flushed and scored at the very end of the capture, so nothing is silently dropped.

Once a window closes, the `FeatureExtractor` boils its packets down into **14 numbers**:

#	Feature	What it captures
1	<code>packet_count</code>	how many packets in the window
2	<code>byte_count</code>	total bytes transferred

#	Feature	What it captures
3	ip_packet_ratio	fraction that are IP packets
4	tcp_packet_ratio	fraction that are TCP
5	udp_packet_ratio	fraction that are UDP
6	icmp_packet_ratio	fraction that are ICMP (ping-like)
7	tcp_syn_ratio	fraction of TCP packets with the SYN flag (connection attempts)
8	tcp_ack_ratio	fraction with ACK (acknowledgements)
9	tcp_fin_ratio	fraction with FIN (graceful close)
10	tcp_rst_ratio	fraction with RST (abrupt reset)
11	packet_length_mean	average packet size
12	packet_length_variance	how much packet size varies
13	inter_arrival_mean_seconds	average time gap between packets
14	inter_arrival_variance_seconds	how much that gap varies

Intuitively: a normal web page load has a handful of packets, a mix of TCP flags, moderate sizes, and fairly regular timing. A port scan looks totally different — almost all SYNs, tiny packets, no replies, extremely regular or extremely bursty timing — which is exactly the kind of thing these 14 numbers make visible to a model, even though the model has no idea what "port scan" means as a word.

2.4 The machine-learning model — Isolation Forest

The core detector is an **Isolation Forest**, a well-known unsupervised anomaly-detection algorithm. The intuition: imagine repeatedly, randomly splitting your data in half (like playing "20 questions" with random questions). A normal, typical data point sits in a dense crowd of similar points, so it takes many random splits to isolate it by itself. An outlier is different from everything around it, so it usually gets isolated in just a few splits. The forest builds many such random-split trees and measures, on average, how quickly each point gets isolated — quick isolation = anomalous, slow isolation = normal.

Before the forest sees any data, a **RobustScaler** rescales all 14 features using the median and interquartile range (rather than mean/standard deviation), so a handful of extreme outliers in the training data can't distort the scale for everything else.

The forest's own raw output isn't a friendly 0–1 score — it's a somewhat arbitrary "decision function" value. So after training, the code:

1. Flips the sign (so *higher* = *more anomalous*, matching how the rest of the app expects it).

2. Finds the **median** and **interquartile spread** of those flipped scores across the training (baseline) data — this becomes the "center" and "scale" of what normal looks like.
3. Applies a **sigmoid** (S-curve) function to squash any future raw score into a clean 0–1 range, centered on that baseline.

One safeguard worth mentioning: if the baseline traffic used for training happens to be very repetitive (e.g. almost all one kind of flow), the natural spread of scores can become almost zero. Dividing by a near-zero spread would make the sigmoid wildly oversensitive — turning ordinary statistical noise into a scary-looking outlier and pushing scores to the extreme for nearly everything, even normal traffic. The code puts a floor (`min_score_scale`, default 0.05) under that spread specifically to prevent this failure mode.

2.5 The bootstrap / training phase — learning what "normal" means

This is an **unsupervised** model: nobody hands it labeled examples of "attack" vs "normal." Instead, the very first **100 completed windows** (across every flow, in the order they finish, config: `bootstrap_windows`) are collected and used, once, to `fit()` the Isolation Forest. During this phase the pipeline is only learning — it does not score anything or raise any alerts.

This makes the quality of that first batch of traffic critically important: whatever the model sees during these 100 windows becomes its definition of "normal," for better or worse. Two consequences that matter a lot in practice, and that came up directly while building this project:

- If the traffic used to warm up the model is dominated by just one kind of flow (say, only web browsing, because it happens to produce far more windows per second than anything else), the model never learns that DNS or SSH traffic is normal too — so the very first DNS or SSH packets it later scores look like total outliers and get flagged as critical, even though they're perfectly ordinary traffic. The fix is to make sure the warm-up traffic is a genuine, balanced mix of all the traffic types you expect to see.
- If anomalous/attack traffic leaks into this warm-up phase, the model partially learns attack patterns as "normal" and gets *worse*, not better, at catching them later. This is why any attack traffic used for testing this system is deliberately kept out of the training/bootstrap phase and only introduced afterward, once the model is already trained.

2.6 The inference engine — turning a score into a decision

Once training is done, every subsequent window goes through the `InferenceEngine`. It's designed to support more than one model at once (an *ensemble*), each with its own weight, combined into one overall score — even though today only the Isolation Forest is registered. For each window:

1. Every registered model scores the window's feature vector (0–1).
2. The scores are combined using each model's normalized weight into one **anomaly_score**.
3. `is_anomalous` is simply `anomaly_score >= threshold` (default threshold: **0.80**).
4. If baseline statistics were captured during training, the engine also works out **contributing_features**: which of the 14 features deviated the most (in standard deviations) from the training baseline. This is what powers the "why was this flagged" explanation shown on the Alert Detail page — instead of just a number, you get a short list like `tcp_syn_ratio`, `inter_arrival_mean_seconds` telling you which measurements looked strangest.

2.7 Alerts — from a score to something you can act on

When a window's score crosses the threshold, the `AlertManager` takes over:

- **Severity tiers** are assigned purely from the score: $\geq 0.95 \rightarrow \text{critical}$, $\geq 0.85 \rightarrow \text{high}$, $\geq 0.70 \rightarrow \text{medium}$, otherwise **low**. (Note the threshold for "is this even an alert" is 0.80, so a 0.80–0.85 score does create an alert, just at "medium" severity — alerting and severity are two separate, independently-configured ideas.)
- **Cooldown / deduplication**: the same flow won't spam a new alert every single window while it stays anomalous — there's a 5-minute cooldown (default) per flow, so one ongoing attack produces one alert, not hundreds.
- **Evidence export**: the exact packets that made up the offending window are written out to their own `.pcap` file (named after the alert's unique ID), so you can open that specific evidence later in Wireshark or replay it, completely separate from the full capture.
- **Persistence**: every alert (with its score, severity, the flow's IP/port info, the 14 feature values, and a path to its evidence file) is saved to a SQLite database (`data/alerts.db`) via a small "sink" abstraction, so the dashboard can query alert history independent of whether a capture session is still running.
- **Alert management** (the feature added most recently): each alert can be marked "observed" (acknowledged) once you've reviewed it, deleted individually (which also deletes its evidence file), or the entire alert history can be emptied in one action, wiping the database rows and every evidence file at once.

2.8 The REST API — FastAPI

The backend exposes everything over HTTP using **FastAPI**, organized into three routers:

- **status** — GET `/status`: is a capture session running, is the model still learning or ready, how many flows are active, the alert threshold, etc. This is what the dashboard polls every 2 seconds.
- **capture** — POST `/capture/start` / POST `/capture/stop`, plus GET `/capture/interfaces`, `/capture/pcaps`, `/capture/models` to list what's available to choose from (network interfaces, saved `.pcap` files, saved trained models).
- **alerts** — GET `/alerts` (list, with filters for severity/IP/observed-status/date range and pagination), GET `/alerts/stats` (counts per severity, for the dashboard's chart), GET `/alerts/{id}` (full detail), GET `/alerts/{id}/evidence` (download the evidence pcap), POST `/alerts/{id}/acknowledge` (mark/unmark observed), DELETE `/alerts/{id}` (delete one), DELETE `/alerts` (empty the whole history).

Starting a capture session doesn't block the API: it's launched on its own background thread (`CaptureController`), so you can still poll `/status` or browse alerts while a capture is actively running. On startup, the app opens one shared alert database connection that lives for as long as the server runs (a FastAPI "lifespan" hook), and cleanly closes both the database and any running capture on shutdown.

2.9 Configuration — tuning the pipeline without touching code

Settings are resolved in four layers, each overriding the one before: **built-in defaults** → **an optional YAML config file** → **environment variables** → **explicit overrides** (like command-line flags or a value sent in an API request). Unknown settings are rejected outright rather than silently ignored, which catches

typos early. The main tunable knobs live under a few sections: capture (interface, filter, where .pcap files live), analysis (the 0.80 alert threshold, 50-packet/30-second window size, the 100-window bootstrap size, flow idle timeout, alert cooldown), model (where trained models are saved/loaded from), storage (where the alert database and evidence files live), and logging (log verbosity).

3. Frontend Walkthrough (React + TypeScript)

3.1 Tech stack and structure

The frontend is a **React + TypeScript** single-page app built with **Vite**, using **React Router** for navigation. `App.tsx` is the shell: it renders a top navigation bar (Dashboard / Alerts) and a `<Routes>` block mapping three URLs to three pages — `/` (Dashboard), `/alerts` (Alerts list), and `/alerts/:alertId` (one alert's detail page). All API calls go through a small typed client (`api/client.ts` + `api/types.ts`), and during development, Vite proxies requests from the frontend's `/api/...` path straight to the FastAPI backend, so the browser never needs to know the backend's real port.

3.2 The Dashboard page

This is the control room. It polls `GET /status` and `GET /alerts/stats` every 2 seconds so the page always reflects what's currently happening, without needing a manual refresh. It shows:

- **Pipeline status:** idle / running / stopping / failed, which source is active, whether the model is still "learning" (bootstrap phase) or ready to score, how many flows are currently active/evicted, and the configured alert threshold.
- **Capture control:** a form to start a session — choose between a **live interface** or a **PCAP file** to replay, an optional BPF filter, promiscuous mode, and whether to **train a new baseline this run** (optionally saving it to a named file afterward) or **load a previously saved model** (skipping training entirely and scoring right away). A Start/Stop button pair drives `POST /capture/start` and `POST /capture/stop`.
- **Alerts by severity:** a simple bar-style chart (`SeverityChart` component) summarizing how many low/medium/high/critical alerts exist right now.

3.3 The Alerts page

A filterable, paginated table of every alert in the database. You can filter by severity, by IP address, and by whether an alert has been "observed" yet. Each row shows the time, severity badge, score, protocol, the two endpoints of the flow, whether evidence exists, and whether it's been observed — with per-row buttons to open its detail page, toggle observed/unobserved, or delete it (with a confirmation prompt, since deleting also removes its evidence .pcap from disk and cannot be undone). A page-level **"Empty all alerts"** button (also confirmed first) wipes the entire alert history and every evidence file at once — this is the feature that was added so old, already-reviewed evidence doesn't have to pile up forever.

3.4 The Alert Detail page

Opening one alert shows everything about it: severity, exact anomaly score, observed status, when it was created, the flow's protocol and both endpoints (with ports), which IPs were flagged as suspicious, a download link for the evidence .pcap (if one was captured), the per-model score breakdown, the list of "contributing features" that most explain why it was flagged, and the full 14-value feature vector for

that window. It also has its own Mark/Unmark-observed and Delete buttons, so you don't need to go back to the list to act on a single alert.

3.5 The API client layer

`api/types.ts` defines TypeScript interfaces that mirror the backend's Pydantic schemas field-for-field (e.g. `AlertOut`, `StatusResponse`), so the compiler catches a mismatch if the two ever drift apart. `api/client.ts` wraps every backend endpoint in a small typed function (`api.getStatus()`, `api.listAlerts(filters)`, `api.deleteAlert(id)`, etc.), all built on one shared `request()` helper that adds the `/api` prefix, sends/parses JSON, and turns a non-2xx HTTP response into a typed `ApiError` with the backend's own error message attached — so every page can just try `{ ... } catch (err) { if (err instanceof ApiError) ... }` and show a clean error instead of a stack trace.

4. End-to-End Journey: One Packet's Life

Say a port scan starts: an attacker's machine sends TCP SYN packets to `172.16.0.50`, one per port, trying ports 1 through 600.

1. **Capture:** each SYN packet is read (live, or from a replayed `.pcap`) and wrapped in a `PacketEnvelope` with a timestamp and sequence number.
2. **Demultiplex:** because each packet targets a different destination port, each one gets its *own* `StreamKey` — a port scan looks like hundreds of brand-new, one-packet "flows" rather than one long conversation.
3. **Window:** each of those single-packet flows never reaches 50 packets and typically doesn't sit open for 30 seconds either, so most of them are only closed out at the very end of the capture, via the pipeline's final "flush everything" step.
4. **Extract features:** each closed window (in this case, usually just 1 packet) becomes a 14-number vector — heavily skewed toward `tcp_syn_ratio = 1.0`, tiny packet size, and (because there's only one packet) zero inter-arrival variance.
5. **Score:** assuming the model has already finished its 100-window bootstrap on ordinary traffic, this feature vector gets compared against the learned baseline. It looks nothing like typical web/DNS/SSH traffic, so it gets a high anomaly score — in testing, consistently around 0.88.
6. **Alert:** since $0.88 \geq$ the 0.80 threshold, `AlertManager` creates an alert, grades it "high" severity (0.85–0.95 band), writes the single triggering packet out to its own evidence `.pcap`, and saves the alert row (score, endpoints, feature values, evidence path) to `data/alerts.db`.
7. **Serve & display:** the FastAPI `alerts` router picks this up automatically the next time the dashboard's Alerts page polls `GET /alerts` — a new row appears with severity "high", the attacker's IP visible in the flow, and a working "Download evidence PCAP" link. Clicking into it shows exactly which of the 14 features (almost certainly `tcp_syn_ratio` and the timing features) drove the score.

5. Key Design Decisions (Good Talking Points)

- **Unsupervised learning, on purpose.** No hand-labeled "attack" dataset is needed — the system learns what's normal from whatever traffic it's shown first. Trade-off: it's only as good as that first batch of "normal" traffic, and it can't yet name *what kind* of attack it saw, only that something didn't fit the pattern.

- **Per-flow windowing, not per-packet scoring.** Scoring individual packets would be noisy and near-meaningless (a single SYN packet tells you almost nothing); aggregating a flow's behavior over a window gives the model actual behavioral patterns to compare.
- **Two window-closing conditions (count OR time), plus a final flush.** This balances chatty flows (closed quickly by packet count) against sparse ones (closed by a time limit) and guarantees nothing — like a single lonely scan probe — is silently lost at the end of a capture.
- **Robust scaling + median/IQR calibration instead of mean/standard-deviation.** Both choices resist being thrown off by a few extreme outliers in the data, which is common in network traffic (a handful of huge transfers, or a burst of scan packets).
- **A floor on the score-scale calibration (`min_score_scale`).** Defends specifically against a baseline that happens to be too uniform, which would otherwise make the sigmoid oversensitive and flag almost everything as extreme.
- **Deliberately keeping attack traffic out of the training/bootstrap phase.** Because the model is unsupervised, anything it sees during training is implicitly taught as "normal." Any test data mixing normal and anomalous traffic is applied only *after* training completes, never during it.
- **Alert cooldown per flow.** Without it, one ongoing attack would generate one alert per window forever, flooding the alert list with duplicates instead of one clear signal.
- **Evidence isolated per alert.** Instead of one giant capture file, each alert gets its own small, focused `.pcap` containing exactly the packets that triggered it — easy to open, easy to delete once reviewed, and safely scoped so deleting an alert can't accidentally touch unrelated evidence.
- **Layered configuration (defaults → YAML → env vars → explicit overrides).** Lets you tune sensitive numbers like the alert threshold or window size without touching code, and rejects unknown settings outright so a typo fails loudly instead of being silently ignored.